A CASE FILE: ANALYTICS ENGINEER COMPANION

The Analytics Engineer's Toolkit

Thirteen reference cards and four fillable templates, pulled from the same cases you'd work if you were Solace's newest Analytics Engineer — genericized so you can actually use them on your own job, today.

By Simran Tiwari · simrant.github.io

BEFORE YOU START

How to use this toolkit

Every card and template here was built to solve a real crisis inside Case File: Analytics Engineer's fictional company, Solace — then stripped of the story so it works on whatever you're actually dealing with.

1. The reference cards (Sections 1–3) are read-only — keep them open next to your editor the way you'd keep a syntax reference open.

2. The four templates (Section 4) are real fillable PDF fields — click into any cell and type, in Acrobat, Preview, or any standard PDF reader.

3. None of these are graded or checked — there's no answer key here, because your own work doesn't have one either. That's the point.

4. If you want the full story behind where these came from — the specific bugs, the specific stakeholders, the specific fixes — Levels 1–4 of Case File: Analytics Engineer are where each of these was actually earned.

SECTION 1 · JUDGMENT & INVESTIGATION

The Curiosity Framework

Three questions to ask before you trust any number — the single most reused instinct across every level of Case File: Analytics Engineer.

Before You Trust Any Number

1. Ask for the definition, not the number.
2. Find the source, not the summary.
3. Assume good faith, expect divergence.

Use this before every metric you're handed in a meeting, a Slack message, or a dashboard you didn't build yourself.

SECTION 1 · FIELD NOTES

Rookie mistakes & pro secrets, part 1

ROOKIE MISTAKE — Reporting a number in a meeting because it's the one you were handed, without asking who calculated it or how.

PRO SECRET — The most trusted Analytics Engineers are the ones who say "let me confirm the definition" out loud, in the room, even when it's uncomfortable. It looks like uncertainty. It's actually the opposite.

Dashboards rot. Nobody budgets time to revisit a query after it ships, and a "temporary" filter becomes permanent silently.

The first thing I do when two numbers disagree isn't pick a side — it's open both queries side by side. Usually one of them is just old.

SECTION 1 · FIELD NOTES

Rookie mistakes & pro secrets, part 2

Before trusting any number after a JOIN, run COUNT(*) before and after adding the join. If the row count jumped and you didn't expect it to, you have a fan-out — full stop.

"1-to-many" isn't a footnote in the data dictionary — it's the single most important fact about any table you're about to join to.

Handy one most people never get told outright: `dbt run --select +model_name` rebuilds a model and everything it depends on, without rebuilding the entire project — the + is the whole trick.

Every "we didn't expect that value" bug has the same root cause: a CASE statement written for the values that existed on the day it was written, not the values that would eventually exist.

SQL Cheat Cards — Filters & Aggregation

SELECT & WHERE Basics

SELECT the columns you need, not SELECT *. WHERE filters rows before any grouping happens.

Positive vs. Negative Filters

Prefer = / IN over != / NOT IN when you know the exact state you want. Negative filters silently let in anything you forgot to exclude.

COUNT(*) vs COUNT(DISTINCT col)

COUNT(*) counts rows. COUNT(DISTINCT user_id) counts people. They stop being the same the moment a table can have more than one row per person.

Date Filters

>= and < are safer than BETWEEN for date ranges. Watch for timezone boundaries and off-by-one month errors.

Business-card sized on purpose in the original — print these, keep them on your desk.

SQL Cheat Cards — JOINs

INNER vs. LEFT JOIN

INNER JOIN drops rows with no match on either side. LEFT JOIN keeps every row from the left table, matched or not — know which one you actually want.

What Is Fan-Out?

Joining to a table where the relationship is 1-to-many multiplies your "one" side by however many matches exist on the "many" side.

Pre-Aggregate First

When the right-hand table is 1-to-many, summarize it into one row per key in a CTE before joining — not after.

Sanity-Check Every JOIN

Compare COUNT(*) before and after adding a join. Any unexpected increase means you just fanned something out.

Pair this with Set 1 — most production bugs live at the intersection of a bad filter and a bad JOIN.

SECTION 3 · DBT REFERENCE

dbt Cheat Cards

staging → intermediate → marts

Staging: 1:1 with a source, cleaned and renamed. Intermediate: business logic combined across staging models. Marts: the finished, wide table dashboards actually query.

ref() vs. source()

source() points to a raw table dbt didn't build. ref() points to another dbt model — dbt uses every ref() to build its dependency graph and run models in the right order automatically.

Materializations

View: always fresh, no storage. Table: materialized, fast to query, needs scheduling. Ephemeral: inlined into whatever references it — no database object at all.

Tests

unique and not_null catch most bugs before a dashboard ever sees them. relationships tests catch orphaned foreign keys. Run tests before you trust a model, not after someone complains.

Remember: {{ }} matters. A ref() without curly braces isn't Jinja — it's just text that won't compile.

SECTION 4 · FILLABLE TEMPLATES

The Metric Audit Worksheet

Use this whenever you're handed a metric you didn't build yourself — a dashboard number, a report figure, anything you're expected to trust or explain. Fill in one row per metric you're auditing.

Metric	Where it comes from	How it's defined	What could make it wrong

Originally earned in Level 1: The Spreadsheet Kingdom

SECTION 4 · FILLABLE TEMPLATES

The Number Reconciliation Worksheet

Use this whenever two people, two dashboards, or two teams report different numbers for the same thing. Fill in one row per discrepancy you're reconciling.

Two conflicting numbers	Where each came from	Which filter/definition differs	Which one you'd trust, and why

Originally earned in Level 2: The SQL Forest

SECTION 4 · FILLABLE TEMPLATES

The One True Customer Table

Use this whenever you're about to JOIN into a table and you're not sure if the relationship is 1-to-1 or 1-to-many. Fill in one row per table you're evaluating.

Table you'd start from	What makes it 1-to-many	How you'd pre-aggregate it	What you'd sanity-check after

Originally earned in Level 3: The JOIN Labyrinth

The Canonical Definition Worksheet

Use this whenever a metric has more than one competing definition floating around your org, and someone needs to end the argument for good. Fill in one row per metric you're canonicalizing.

Metric with competing definitions	Where each definition currently lives	What the canonical definition should be	Who needs to sign off before it ships

Originally earned in Level 4: The dbt Factory

WHERE THESE CAME FROM

Every card here was earned, not invented

Each reference and template in this toolkit is the exact tool used to solve a real production crisis inside **Case File: Analytics Engineer** — a case-file-style interactive simulation where you're the newest Analytics Engineer at Solace, a startup whose data has quietly fallen apart. If you want the story behind where these came from — the specific bugs, the specific stakeholders, the specific fixes — that's where to find it.

Level 1 · The Spreadsheet Kingdom

Level 2 · The SQL Forest

Level 3 · The JOIN Labyrinth

Level 4 · The dbt Factory

Level 1 is free, no email required. simrant.github.io